Fiche d’investigation de fonctionnalité

Fonctionnalité: Recherche	**Fonctionnalité #2**
Problématique: Permet de trier parmis les recettes disponibles via un champ de recherche et des menus de sélections d’ingrédients, d’appareils et d’ustensils à partir de 3 caractères. La recherche renverra alors les recettes dont le nom, la description ou les ingrédients qui sont contenu dans la recherche, et dont ceux-ci correspondent aux menus de sélections. **Architecture:** Séparé de l’application front end afin d’améliorer la maintenabilité, de permettre son utilisation par plusieurs front end, et de ne pas être dépendant de l’interface du front end. Pourrait à l’avenir être accessible via une API.	
Nombre de champ: 1 champ recherche (optionnel) **Nombre de sélecteur:** 3 sélecteurs (Ingrédients, ustensil, appareil) (optionnels) **Nombre de champ minimum:** 0 champ et 0 sélecteur (renvoie alors toutes les recettes)	

Option 1: Utilisation d'une FOR loop Dans cette approche, nous utilisons un for loop.	
Avantages: ● Simple à comprendre ● La plus performante ● Optimisation	**Inconvénients:** ● Moins facile à lire ● Difficilement maintenable

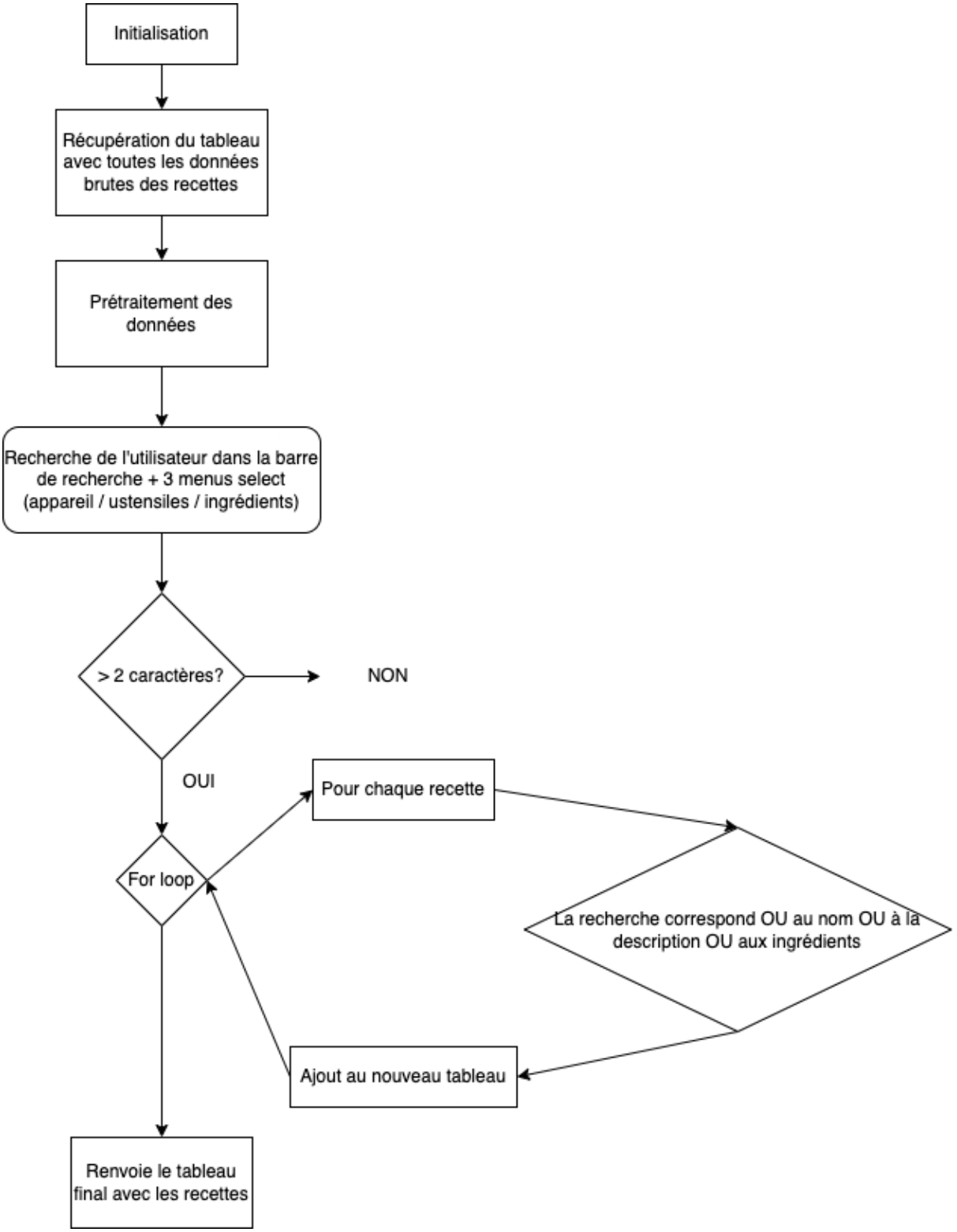
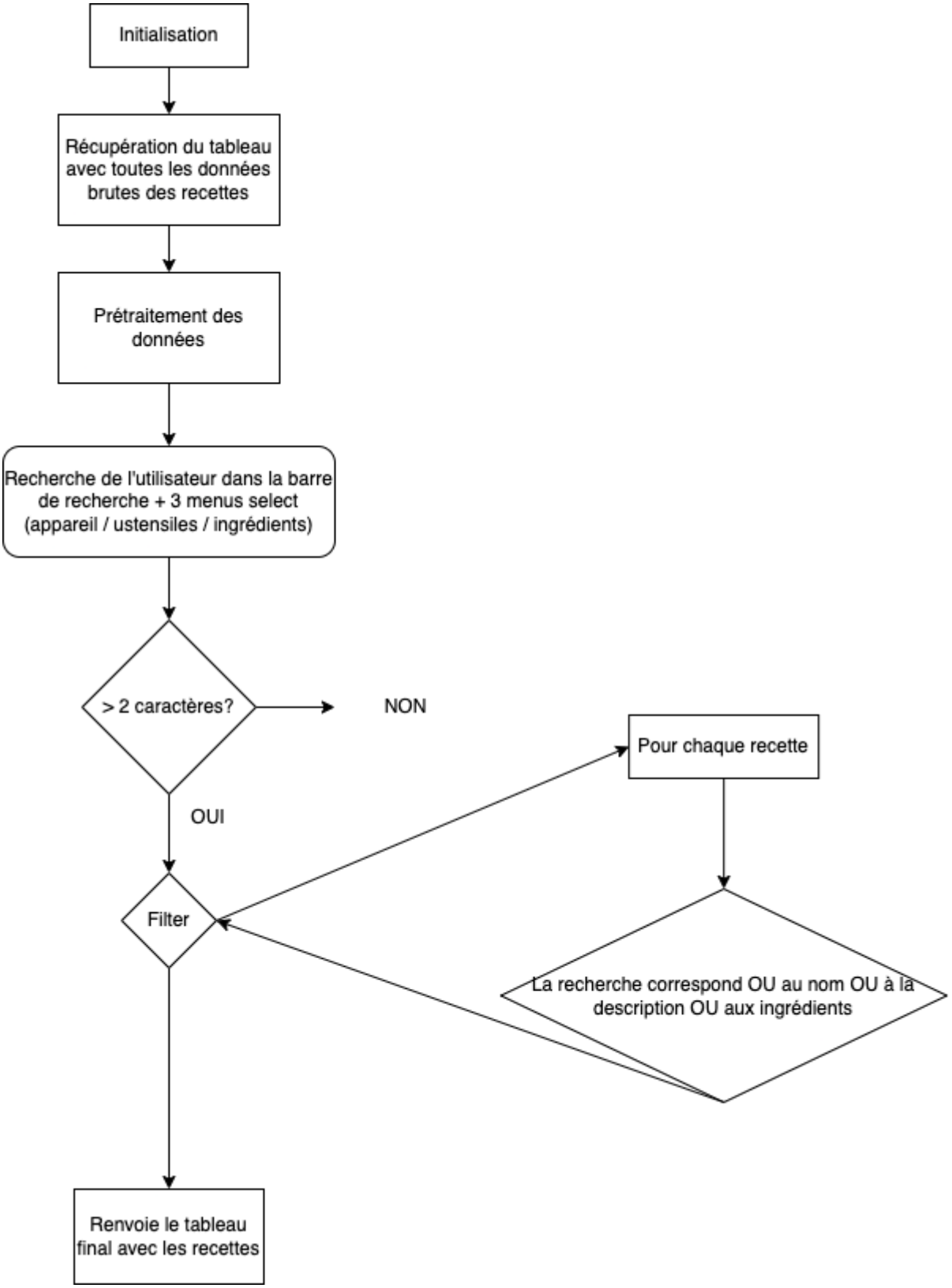
Option 2: Utilisation d'une fonction Filter Dans cette approche, nous utilisons une fonction native Filter de Javascript.	
Avantages: ● Simple à coder ● Code facile à comprendre ● Implementation très facile.	**Inconvénients:** ● Moins performant à cause du callback ● Surcharge des actions à effectuer (getters, sparse array et verification des arguments)

Solution retenue: Option 1, car on nous demande de la performance

Resultat des tests entre Js Native / For loop / For each et Lodash :

For large data se	50000-1000000			
Array Size	50000			
Function	Reduce	Map	Filter	Find
Methods	Time (ms)	Time (ms)	Time (ms)	Time (ms)
JS native	4.773	14.534	1.59	1.022
For loop	3.463	1.404	1.711	1.264
For each	3.192	1.668	1.641	2.205
Lodash	3.582	1.659	1.638	1.239
Array Size	100000			
Function	Reduce	Map	Filter	Find
Methods	Time (ms)	Time (ms)	Time (ms)	Time (ms)
JS native	7.597	29.347	3.207	1.902
For loop	7.511	2.809	2.034	1.357
For each	6.237	10.842	3.376	3.156
Lodash	6.694	4.415	2.649	1.273
Array Size	500000			
Function	Reduce	Map	Filter	Find
Methods	Time (ms)	Time (ms)	Time (ms)	Time (ms)
JS native	36.909	129.026	13.839	9.094
For loop	29.783	4.806	5.882	3.762
For each	31.79	12.79	42.116	11.521
Lodash	30.773	8.224	9.659	4.225
Array Size	1000000			
Function	Reduce	Map	Filter	Find
Methods	Time (ms)	Time (ms)	Time (ms)	Time (ms)
JS native	71.219	276.729	29.769	17.282
For loop	58.245	8.491	12.704	6.772
For each	64.462	28.129	31.082	23.71
Lodash	60.515	16.812	15.938	5.859

OPTION 1: FOR LOOP



OPTION 2: FONCTION FILTER